SZY v8 — engineering documentation of the engine, format, and tests
Lossy compression of scientific 2D data: transmission, archiving, fast decode

Project: SzKompresor / SZY v8

May 1, 2026

Contents

1 Introduction: lossy compression in science and data integrity **3**
- 1.1 Lossy vs lossless codecs . . . 3
- 1.2 Engine vs application (the role of the client) . . . 3
- 1.3 Why integrity is part of the format . . . 3

2 Goal and scope of this document (what is "complete") **3**

3 SZY v8 as an engine and format: applications and niches **4**
- 3.1 What SZY v8 is (engineering definition) . . . 4
- 3.2 Where SZY v8 makes scientific/technical sense . . . 4
- 3.3 Why comparisons to leaders are necessary even for a niche . . . 6
- 3.4 Differences vs SZ3 and ZFP (what we compare and what we do not) . . . 6
- 3.5 Lossless as a baseline, not a goal . . . 7

4 Project structure and modules **7**
- 4.1 Module overview (complete short form) . . . 7
- 4.2 Key library structures (C API) . . . 8
- 4.3 Python wrapper: `SzyDLL` class . . . 9

5 Metrics and definitions **10**
- 5.1 Size, ratio, bpp . . . 10
- 5.2 Errors . . . 10
- 5.3 Data scale vs eps (important in benchmarks) . . . 10

6 Portable specification: types, endianness, varuint **10**
- 6.1 Endianness . . . 10
- 6.2 Varuint . . . 10

7 SZ2D v8 container format — full specification **10**
- 7.1 Container layout (normative) . . . 10
- 7.2 Meaning of `flags` . . . 11
- 7.3 Index blob . . . 11
- 7.4 Integrity (SHA) and error codes . . . 11
- 7.5 Requirements and validations (reference implementation) . . . 11

8 Tile record v8 format — full specification **12**
8.1 IDs and flags . . . 12
8.2 Tile record layout (normative) . . . 12
8.3 Events: indices and amplitudes . . . 13
8.4 Payload and expected decompressed size . . . 13

9 Algorithm: encode/decode (in detail) **13**
9.1 Encode: per-tile steps . . . 13
9.2 Decode: per-tile steps . . . 14

10 Parallelism: tile-based decode and encode **14**
10.1 Parallel decode . . . 14
10.2 Parallel encode . . . 14

11 Engine API (DLL) and error codes **15**
11.1 Error codes . . . 15
11.2 Memory rules (ABI) . . . 16
11.3 Threading policy . . . 16
11.4 API error semantics . . . 16
11.5 Selected DLL exports (engine) . . . 16
11.6 DLL function descriptions (what they do and how to use them) . . . 17

12 Build (Windows/MSVC) **17**

13 Engineering tests and benchmarks: methodology **18**
13.1 Artifact identification (DLL SHA256) . . . 18
13.2 Two test levels: synthetic auto-suite and public benchmark . . . 18
13.3 Public benchmark (bench_public.py): data and codecs . . . 18
13.4 Auto-suite v1: synthetics + MNIST/Fashion-MNIST (older pipeline) . . . 19

14 Results (manual): granule 10880×10880 float64 **19**
14.1 Comparison with SZ3/ZFP and lossless baseline . . . 19

15 Results (auto-suite v1): synthetics + MNIST/Fashion-MNIST (older experiment) **20**
15.1 Results on downloaded data (MNIST/Fashion-MNIST): summary table . . . 20

16 Results (public benchmark v3): satellite, sensors, synthetic data **20**
16.1 Selected datasets and configurations . . . 20
16.2 Scientific/technical interpretation of results . . . 21

17 Main conclusions (v8 state after tests) **22**
17.1 What works and what is the engine value . . . 22
17.2 What comparisons to SZ3/ZFP teach (without changing the niche) . . . 22
17.3 Development recommendations (consistent with the tile-based engine philosophy) . . . 22
17.4 Scientific potential and practical advantages . . . 23

A Appendix A: Artifacts and reproducibility **24**

B Appendix B: Minimal compliance specification (must-pass) **24**

References **25**

1 Introduction: lossy compression in science and data integrity

1.1 Lossy vs lossless codecs

A lossy codec allows a controlled deviation of the output data from the input data in exchange for a smaller size. In a scientific context, the key is an **explicitly defined error model**, not subjective quality.

Typical models:

- **Bounded L∞ (absolute error bound)**: $\max_i |x_i - \hat{x}_i| \leq \varepsilon$.
- **Relative bound**: $\max_i \frac{|x_i - \hat{x}_i|}{|x_i| + \delta} \leq \varepsilon_{rel}$.
- **Energy measures**: RMSE, L2, PSNR as additional characterization.

Note on "rounding after the 12th decimal place". In many scientific applications, an error on the order of 10^{-12} (for data scaled around ≈ 1) is acceptable. One must remember, however, that the meaningfulness of such a tolerance depends on the *scale* and units of the data: for values on the order of 10^6 or 10^7, the `float64` representation precision (ULP) may be larger than 10^{-12}.

1.2 Engine vs application (the role of the client)

SZY v8 is an **engine**. This means:

- the **engine provides the mechanics** (format, pipeline, integrity, threads, decode-into),
- the **client selects parameters** (e.g., `eps`, block `B`, number of workers, quantization mode, event policy),
- "scientific" correctness is evaluated in the context of *real data* and the client's requirements,
- for data classes *not matching the model*, the engine may explicitly refuse to satisfy the requested bounded-error instead of returning a result that violates the guarantee.

The tests in this document show that with sensibly chosen parameters for image-like and noisy data classes, SZY meets quality expectations (controlled error) and offers favorable trade-offs relative to reference codecs. At the same time, the benchmark also includes *stress-test / out-of-domain* cases whose purpose is not to prove the superiority of the model but to establish the boundaries of applicability.

1.3 Why integrity is part of the format

In practice, there is a risk of **silent data corruption**. In SZY v8, integrity is part of the container:

- `data_sha`: SHA-256 of the data section (index + tile records),
- `container_sha`: SHA-256 of the entire container.

This makes it possible to detect damage and unambiguously return errors: `SZY_E_SHA_DATA` and `SZY_E_SHA_CONT`.

2 Goal and scope of this document (what is "complete")

This document includes:

1. The **binary specification** of the SZ2D v8 container and v8 tile record (layout, types, varuint, endianness, SHA).

2. A **description of the algorithm** for encode/decode (the transformation pipeline and its ordering).
3. A **description of the engine**: DLL/ABI API, thread control, decode-into, index, integrity.
4. **Build** (MSVC) and dependencies (zstd/zlib).
5. **Engineering tests** (manual, public benchmark): methodology, datasets, results, observations.
6. **Positioning vs leaders** (SZ3 and ZFP): differences in philosophy, what comparisons mean, conclusions.

Note: `eps=0` does not mean lossless. In v8, `eps=0` disables the bounded-error model, but it does **not** disable quantization (it is still lossy compression). Lossless mode is not a goal of v8.

3 SZY v8 as an engine and format: applications and niches

3.1 What SZY v8 is (engineering definition)

SZY v8 is a **tile-based bounded-error codec** for 2D data and a **container** with integrity and an index, exposed as a **C library / DLL**.

Key engine features:

- **Tile independence**: the image is split into $B \times B$ tiles; each tile record is independent. This enables:
  - local compression/decompression (region-of-interest),
  - straightforward parallelism over tiles,
  - resilience to local damage (damage to one tile does not invalidate the whole).
- **Index blob**: a list of tile lengths (varuint), enabling fast computation of tile offsets and balanced work distribution across threads.
- **Parallel decode**: tile decoding in parallel based on the index, with control of worker count and `parallel_min_tiles` threshold.
- **Decode-into**: decompress directly into a user-provided buffer (without allocating an additional output image) — important for HPC/FORTRAN codes that manage memory themselves.
- **Integrity**: SHA-256 of data and container as part of the format; the user does not need to implement checksums.
- **Type support**: float64, float32, uint16 — covers typical scientific data (climate fields, sensors, image intensities).
- **Cross-platform**: little-endian format, C implementation, can run on x86-64 and ARM (including edge/embedded).

3.2 Where SZY v8 makes scientific/technical sense

The engine is especially useful in the following scenarios:

Climate and oceanography (NetCDF, geophysical grids).

- 2D fields (SST, near-surface temperature, pressure) from climate models or reanalyses,
- horizontal slices from 3D simulations (ROMS, NEMO, ICON),
- data streams from catalog services (THREDDS, pyDAP) after conversion into RAW C-ordered blocks.

For such data, the key is:

- preserving field shape (gradients, mesoscale structures),
- error control (so as not to break sensitive assimilation/diagnostics),
- throughput (HBM/SSD/ethernet).

The benchmark shows that on a synthetic climate field (f32) SZY reaches a ratio of 10^3 at eps=10–20 and meets the bound.

Satellite imagery and optical sensors.

- GeoTIFF/CCSDS extracted as uint16,
- optical sensors (Landsat, Sentinel) reduced to a single band,
- downlink and archiving at ground stations.

For this data class, SZY v8:

- yields ratio ∼8–16x at eps=5–20, while Zstd/Blosc2 ∼4.4x,
- provides a hard error bound (important for downstream processing, e.g., vegetation indices),
- allows partial image decode (tile-based).

Medical (CT) and astronomical (CCD) data.

- Example synthetic CT/CCD in uint16 format (simulating HU or ADU),
- Zstd/Blosc2 offer ∼1.7–1.8x, while SZY at eps=20 reaches 5–6x (with error control).

SZY is a natural candidate for long-term archiving: eps can be chosen so that the error is substantially smaller than the noise/instrument variance.

Sensor streams (UCI: AirQuality, Energy) and time series.

- For `sensor_energy_uci` at eps=20, B=128: SZY gives 532x at max_err≈19.6; SZ3 gives 470x, ZFP ∼126x, Zstd ∼38x.
- For `sensor_airquality_uci` at eps=20, B=128: SZY and SZ3 are in the same ratio class (196x vs 206x), SZY is faster.

This suggests potential for SZY as a building block in IoT/telemetry pipelines where:

- streams are larger than raw storage capacity,
- a small explicit measurement error is acceptable,
- encoder/decoder speed on laptop/edge-class CPU is important.

HPC simulations (CFD, turbulence, mesoscale phenomena).

- 2D cross-sections from HDF5/NetCDF files (e.g., Hurricane, Miranda),
- checkpoint dumps into an archive (tile-encoded),
- partial reads via decode-into (an MPI rank reads only its own fragment).

Here SZY v8 provides not only ratio but also an engine: easy integration into existing Fortran/C++ codes (C library, decode-into buffer).

3.3 Why comparisons to leaders are necessary even for a niche

Even if you build a niche, you still need comparisons to leaders, because:

- the market understands SZ3 and ZFP as reference points (scientific codec vs "image codec"),
- a benchmark establishes the trade-off map: bitrate vs error vs time,
- comparisons reveal data classes where it is worth adding optional modes (e.g., trend, other predictors).

Note on out-of-domain benchmarks. The tests also include synthetic data with structure that is extremely unfavorable for the current model (e.g., sharp discontinuities at tile boundaries). These are not target data, but they are useful as a **boundary benchmark**: they show when the method loses effectiveness or should explicitly return a bounded-error failure rather than pretending success.

3.4 Differences vs SZ3 and ZFP (what we compare and what we do not)

ZFP (a specialized solution). ZFP [2] is specialized for compressing FP arrays (block-based, with fixed-rate/fixed-precision/fixed-accuracy modes). It is an excellent reference as a *pure floating-point codec*, but:

- it is not a container-engine (no SHA, no index, no decode-into as standard),
- it does not support uint16 as a first-class type (conversion required).

In benchmarks, SZY v8 very often **beats ZFP at the same eps** in ratio, especially on sensor and image-like data.

SZ3 (a framework / often maximizes bitrate). SZ3 [3] is a modular framework that often achieves very high ratios on data classes matching its strategy (especially very smooth fields). It is a good reference as an "upper bound" on bitrate under a given error model. On the other hand:

- it is complex to configure and integrate (multiple modes, pipelines),
- it does not include a built-in container with SHA and tile index,
- it can be slower than SZY v8 on the same datasets at the same eps.

SZY v8 (positioning). SZY v8 optimizes simultaneously:

- bounded $L\infty$ (controlled error),
- encode/decode speed and MT decode,
- integrity and robustness (SHA),
- integration ergonomics (decode-into, peek-shape, C API, Python wrapper),

- uint16 support (important for images).

From benchmarks:

- on smooth climate fields, SZ3 has better bitrate (2–3×),
- on real sensor data, SZY can beat SZ3 in ratio at the same eps (e.g., `sensor_energy_uci`),
- SZY consistently beats ZFP at the same eps and is much better than the lossless baseline for most datasets.

3.5 Lossless as a baseline, not a goal

The tests include lossless codecs (Zstd, Blosc2) as a **baseline**:

- they show how much redundancy is present without loss,
- they help decide whether lossy compression is worth it for a given data class,
- they are not a design target of SZY v8 (SZY is designed for lossy scientific).

4 Project structure and modules

Example directory layout:

```
src/
  szy_bitplane.h, szy_bitplane.c
  szy_buf.h,  szy_buf.c
  szy_bytes.h, szy_bytes.c
  szy_endian.h
  szy_varuint.h, szy_varuint.c
  szy_sha256.h, szy_sha256.c
  szy_container_v8.h, szy_container_v8.c
  szy_tile_v8.h, szy_tile_v8.c
  szy_parallel.h, szy_parallel.c
  szy_decode_v8.h, szy_decode_v8.c
  szy_encode_v8.h, szy_encode_v8.c
  szy_encode_f32.c
  szy_encode_u16.c
  szy_hw.h, szy_hw.c
  szy_rc.h
  szy_errors.c
  szy_exports.c
include/
  szy_api.h
  szy_dll.h
```

4.1 Module overview (complete short form)

Note on the `.[ch]` notation. This documentation uses the shorthand `foo.[ch]` meaning a **pair of files** `foo.c` (implementation) and `foo.h` (declarations). This is a technical convention, not an actual file extension.

- `szy_bitplane.[ch]`: helper module for 16-bit (u16) bitplane transforms — the current version implements `szy_bitplane_shuffle_u16()` and `szy_bitplane_unshuffle_u16()`, ready for use in the pipeline (the v8 C encoder uses shuffle16 by default, but bitplane is fully implemented and may be enabled in future API versions).

- `szy_buf.[ch]`: growable buffer writer, LE type writes.
- `szy_bytes.[ch]`: reader/cursor for safe parsing (bounds checking).
- `szy_endian.h`: portable store/load for u16/u32/u64 and f32/f64 in LE.
- `szy_varuint.[ch]`: varuint decode/encode (7-bit payload + MSB continuation).
- `szy_sha256.[ch]`: SHA-256 implementation (no external dependencies).
- `szy_container_v8.[ch]`: v8 container parsing, SHA verification, reading the tile-length index.
- `szy_tile_v8.[ch]`: decoding a single tile record (entropy + undo transforms + predictor).
- `szy_parallel.[ch]`: parallel tile decode based on the index (Windows/POSIX).
- `szy_decode_v8.[ch]`: full-image decode (output allocation) and parallel/sequential selection logic.
- `szy_encode_v8.[ch]`: full-image encode, container building, events, entropy (ZSTD/-ZLIB/NONE), transforms (Lorenzo, bias, zigzag, shuffle16/bitplane).
- `szy_encode_f32.c, szy_encode_u16.c`: type wrappers (cast into the f64 pipeline).
- `szy_hw.[ch]`: `szy_hw_threads()` + `num_workers` validation (strict/relaxed).
- `szy_exports.c`: DLL exports, `*_ex` API (threads controlled externally).
- `szy_errors.c`: `szy_strerror()`.

4.2 Key library structures (C API)

For C/C++ clients, the following types matter:

`szy_enc_cfg_t.` Encoder configuration struct:

```c
typedef struct {
  int block;                  /* e.g., 64/128 */
  int qbits;                  /* 8 or 16 */
  szy_quant_mode_t qmode;     /* strict/robust */
  double qpercentile;         /* e.g., 99.5 for robust */

  /* bounded error model */
  double target_abs_err;      /* eps; <=0 => bounded-error off */
  double bounded_safety;      /* e.g., 0.98 */
  double quant_err_factor;    /* e.g., 2.0 */

  /* events */
  int events_on;              /* 0/1 */
  int events_max_per_tile;    /* e.g., 2048 */
  double events_max_frac;     /* e.g., 0.01 */

  /* entropy */
  szy_entropy_t ent;          /* ZSTD/ZLIB/NONE */
  int zstd_level;             /* 1..22 */

  /* transforms */
  int use_zigzag;
  int byte_shuffle_16;
  int use_bitplane_shuffle;   /* infrastructure ready; default 0 */

```

```
  /* optional post-factum verification */
  int verify_after_encode;   /* 0/1 */
} szy_enc_cfg_t;
```

A typical configuration used by the DLL sets:

- `qbits=16`, `qmode=strict`, `target_abs_err=eps`,
- `events_on=1`, with an appropriate event budget,
- ZSTD entropy (lvl=3),
- zigzag + shuffle16 (bitplane off in v8 DLL).

`szy_container_hdr_t`. Container header struct after `szy_container_unpack_v8()`:

```
typedef struct {
  uint16_t H, W, block;
  uint8_t version;
  uint8_t flags;

  int index_present;
  uint32_t ntiles;
  uint32_t* tile_lengths; /* optional; malloc'd */

  size_t data_start;
  size_t tiles_start;
  size_t data_end;
} szy_container_hdr_t;
```

Used internally by the decoder and by `szy_peek_shape_v8`.

`szy_bw_t` (writer) and `szy_rd_t` (reader). Simple structs for binary write/read with bounds checking and easy buffer management:

```
typedef struct {
  uint8_t* p;
  size_t n;
  size_t cap;
} szy_bw_t;

typedef struct {
  const uint8_t* p;
  size_t n;
  size_t pos;
} szy_rd_t;
```

4.3 Python wrapper: `SzyDLL` class

Benchmarks use a simple ctypes wrapper:

Listing 1: SzyDLL class (Python, bench$_{c}$*ommon.py*)

```
class SzyDLL:
    def __init__(self, dll_path: str):
        self.dll_path = os.path.abspath(dll_path)
        self.lib = ctypes.CDLL(self.dll_path)
        # map DLL functions to ctypes signatures (...)
    def compress(self, x: np.ndarray, eps: float, block: int, num_workers: int) -> bytes:
        # pick szy_compress_*_ex depending on dtype (f64/f32/u16)
```

```
        # return container bytes
    def decompress_into_f64(self, blob: bytes, out: np.ndarray,
                            num_workers: int, parallel_min_tiles: int) -> (int, int):
        # call szy_decompress_into_buffer_ex2
```

Thus, Python benchmarks see SZY as a "normal" codec (like zfpy, pysz, zstandard), while the entire engine (SHA, index, decode-into) remains in C.

5 Metrics and definitions

5.1 Size, ratio, bpp

- **ratio** $= \frac{\text{raw_bytes}}{\text{compressed_bytes}}$
- **bpp** (bits-per-pixel) $= \frac{8 \cdot \text{compressed_bytes}}{H \cdot W}$

5.2 Errors

- **max_err** $= \max_i |x_i - \hat{x}_i|$
- **RMSE** $= \sqrt{\frac{1}{N} \sum_i (x_i - \hat{x}_i)^2}$
- **p99_err** $=$ 99th percentile of $|x_i - \hat{x}_i|$

5.3 Data scale vs eps (important in benchmarks)

The auto-suite and public benchmark report p01/p99/std. In practice:

$$\texttt{eps_over_range} = \frac{\varepsilon}{p99 - p01}$$

enables comparing "eps aggressiveness" across datasets of different scales.

6 Portable specification: types, endianness, varuint

6.1 Endianness

In the container and tile record, fixed-width numbers are encoded in **little-endian**: `u8, u16le, u32le, u64le, i16le, f32le, f64le`. Floats are IEEE-754.

6.2 Varuint

Varuint encodes `uint64` as a sequence of bytes:

- each byte carries 7 bits of payload,
- MSB=1 indicates continuation,
- MSB=0 terminates the integer.

7 SZ2D v8 container format — full specification

7.1 Container layout (normative)

Table 1: SZ2D v8 container: binary layout (field order).

Field	Type	Size	Description
magic	bytes	4	ASCII `"SZ2D"`
version	u8	1	value `8`
flags	u8	1	bit0: `index_present`
H	u16le	2	image height
W	u16le	2	image width
B	u16le	2	tile size (block)
meta_len	u16le	2	metadata length
meta	bytes	meta_len	blob (e.g., JSON)
data_sha	bytes	32	SHA256(`data_section`)
data_section	bytes	var	`index_blob || tiles_blob`
container_sha	bytes	32	SHA256(container without this field)

7.2 Meaning of `flags`

- `flags & 1` = 1: the tile index is present at the beginning of `data_section`.
- `flags & 1` = 0: no index; tile records are stored sequentially.

7.3 Index blob

If the index is present, `data_section` begins with:

- `ntiles` (varuint),
- `tile_len[i]` (varuint) for i=0..ntiles-1,
- then `tiles_blob`: concatenated tile records.

7.4 Integrity (SHA) and error codes

- `SZY_E_SHA_CONT`: container SHA256 mismatch,
- `SZY_E_SHA_DATA`: data-section SHA256 mismatch.

7.5 Requirements and validations (reference implementation)

The C parser verifies, among other things:

- `magic="SZ2D"` and `version=8`,
- `H,W,B>0`,
- correct data-section boundaries,
- index consistency (sum of tile lengths),
- SHA match.

8 Tile record v8 format — full specification

Note: format specification vs current C encoder implementation. The tile record v8 specification includes fields and modes that the reference decoder understands for compatibility and extensibility. This does not mean that the current C encoder generates all variants. In particular, the current C encoder:

- uses the Lorenzo2D predictor,
- does not generate trends other than `TREND_NONE`,
- includes infrastructure for shuffle16 *and* bitplane shuffle for qbits=16, but the v8 DLL uses shuffle16 by default; bitplane can be enabled in experimental builds,
- no longer generates `amp_mode=2` (float16) for new streams (the decoder still accepts mode 2 only for backward compatibility).

8.1 IDs and flags

Entropy ID (`ent_id`).

- 0: ZLIB
- 2: NONE
- 3: ZSTD

Predictor ID (`pred_id`).

- 0: Lorenzo2D (used by the v8 C encoder)
- (the decoder also supports DIFFX/DIFFY for compatibility)

Tile flags (`flags`).

- bit0 (1): SHUFFLE16
- bit1 (2): ZIGZAG
- bit2 (4): BITPLANE (decoder supports; v8 DLL C encoder does not enable by default, but the shuffle/unshuffle code is compiled in)

8.2 Tile record layout (normative)

Table 2: Tile record v8: binary layout (field order).

Field	Type	Size	Description
th	u16le	2	tile height (should equal B)
tw	u16le	2	tile width (should equal B)
trend_id	u8	1	0=none (v8 C encoder)
qbits	u8	1	8 or 16
qmode	u8	1	0=strict, 1=robust (decoder ignores)
events_id	u8	1	0=off, 1=on
ent_id	u8	1	0=zlib,2=none,3=zstd
flags	u8	1	shuffle/zigzag/bitplane
pred_id	u8	1	0=Lorenzo2D

Field	Type	Size	Description
trend_params_len	u8	1	0/3/6 (number of f64)
trend_params	f64le[]	8*len	trend parameters
inv_scale	f64le	8	quantization scale
mean	f64le	8	tile mean
clip_rate	f32le	4	informative statistic
bias	i16le	2	median bias of deltas
has_pred_params	u8	1	if 1: 16B reserved
pred_params	bytes	16	reserved
ne	varuint	var	number of events
idx_len	varuint	var	length of idx_bytes
idx_bytes	bytes	idx_len	indices (delta-coded varuint)
amp_mode	u8	1	amplitude format
amp_blob	bytes	var	event amplitudes per mode
payload_len	varuint	var	payload length
payload	bytes	payload_len	entropy-compressed data

8.3 Events: indices and amplitudes

Indices. Encoded as varuint deltas (delta coding) over indices in the flattened tile (row-major).

Amplitudes. `amp_mode` defines the representation of event amplitudes:

- 0: scale (f64le) + `ne` int8 bytes,
- 1: scale (f64le) + `ne` int16le values,
- 2: `ne` float16 values (u16le) — legacy, decode-only,
- 3: `ne` float32le values.

Current state of the v8 C encoder. The current implementation **no longer generates mode 2 (float16)** for new streams, because for large amplitudes it could lead to non-finite values and unstable reconstruction. The decoder still accepts `amp_mode=2` solely for **backward compatibility** with older files.

Meaning of events for bounded-error. Events represent local deviations that cannot be efficiently handled by quantization and prediction alone under the requested `eps`. If the number of required events exceeds the budget imposed by client configuration, the engine should not "pretend success"; it should return a bounded-error failure (see the error-code section).

8.4 Payload and expected decompressed size

After entropy decompression the payload must have size:

$$\texttt{expected} = \texttt{npix} \cdot \frac{\texttt{qbits}}{8}, \quad \texttt{npix} = \texttt{th} \cdot \texttt{tw}.$$

9 Algorithm: encode/decode (in detail)

9.1 Encode: per-tile steps

For a $B \times B$ tile (npix=B^2):

1. Compute the mean μ and center the tile: $c_i = x_i - \mu$.

2. (Optional) events: extract sparse deviations $|c_i|$ above a threshold into a list (idx+amp), if they fit within the event budget imposed by configuration.
3. Choose `inv_scale` for quantization (depending on mode and eps).
4. Quantize to int8/int16: $q_i = \text{round}(c_i \cdot inv_scale)$, with clipping.
5. Lorenzo2D prediction on $q \to$ deltas d (mod 2^{qbits}).
6. Median bias of deltas: $d'_i = d_i - bias$ (bias computed as signed median).
7. (Optional) zigzag: signed $\to$ unsigned.
8. (For qbits=16) shuffle16 or bitplane shuffle (if enabled).
9. Entropy coding: ZSTD/ZLIB/NONE.
10. Write tile record: header + parameters + events + payload.

Important property of the current implementation. If in bounded-error mode the number of required events exceeds limits (`events_max_per_tile`, `events_max_frac`), the encoder **returns** `SZY_E_BOUNDS`. Additionally, the implementation contains an internal per-tile fallback mechanism (e.g., relax eps, disable events), but from the perspective of the external contract, **bounded-error comes first**: if it cannot be maintained, it is better to return an error than to violate the bound.

9.2 Decode: per-tile steps

1. Read the tile record.
2. Decompress payload (ZSTD/ZLIB/NONE) $\to$ body of size `expected`.
3. Undo shuffle16 or bitplane (if flagged).
4. Undo zigzag (if flagged).
5. Bias: $d_i = d'_i + bias$.
6. Inverse predictor (Lorenzo2D) $\to q_i$.
7. Dequantize: $c_i \approx q_i / inv_scale$.
8. Reconstruct: $\hat{x}_i = c_i + \mu + trend(i, j)$.
9. Add events (if present): $\hat{x}_{idx_k} \mathrel{+}= amp_k$.

10 Parallelism: tile-based decode and encode

10.1 Parallel decode

If the index is present, the engine computes tile offsets based on lengths and launches workers. In the reference implementation:

- POSIX: a simple thread-pool (pthreads),
- Windows: a fixed worker pool, not "thread per tile".

10.2 Parallel encode

Encode can be done in parallel over tiles; then tile records are concatenated in deterministic order (ti=0..ntiles-1) to preserve a repeatable container.

11 Engine API (DLL) and error codes

11.1 Error codes

Listing 2: Error codes (szy_rc.h)

```
typedef enum {
  SZY_OK           = 0,
  SZY_EINVAL       = -1,
  SZY_E_SHA_CONT   = -2,
  SZY_E_SHA_DATA   = -3,
  SZY_E_INDEX      = -4,
  SZY_E_DIMS       = -5,
  SZY_E_BLOCK      = -6,
  SZY_E_TILECOUNT  = -7,
  SZY_E_OVERFLOW   = -8,
  SZY_E_OOM        = -9,
  SZY_E_TILE       = -10,
  SZY_E_THREADS    = -11,
  SZY_E_BOUNDS     = -12
} szy_rc_t;
```

Error code descriptions (practical meaning). Names and numeric values alone are insufficient in user-facing documentation; the table below describes *what an error means* and *when it typically occurs*.

Table 3: Meaning of SZY v8 error codes (API contract).

Code	Symbol	Class	Meaning / typical causes
0	`SZY_OK`	OK	Success. Operation completed correctly.
-1	`SZY_E_INVAL`	Input/Format	Invalid argument or damaged/incompatible format (e.g., wrong magic/version, invalid tile fields, incorrect lengths, invalid payload).
-2	`SZY_E_SHA_CONT`	Integrity	SHA256 mismatch of the container (container corruption or data mismatch).
-3	`SZY_E_SHA_DATA`	Integrity	SHA256 mismatch of the data section (index+tile blob).
-4	`SZY_E_INDEX`	Format/Index	Tile index is inconsistent (length sums, offsets beyond `data_end`, index blob corruption).
-5	`SZY_E_DIMS`	Input	Invalid `H/W/B` dimensions (e.g., 0 or over limits).
-6	`SZY_E_BLOCK`	Input	Dimensions not divisible by block `B` (in decode) or other block-condition failure.
-7	`SZY_E_TILECOUNT`	Format	Mismatched tile count (`ntiles`) relative to `H/W/B`.
-8	`SZY_E_OVERFLOW`	Safety	Size overflow (e.g., in offset/allocation computations).
-9	`SZY_E_OOM`	Resources	Out of memory (allocation failed).
-10	`SZY_E_TILE`	Decode	Tile decode failure (e.g., entropy decompression size mismatch, unsupported variant, inconsistent tile fields).
-11	`SZY_E_THREADS`	Config	Requested thread count exceeds available hardware threads (strict mode).
-12	`SZY_E_BOUNDS`	Quality/Bound	Controlled compression refusal: with the current bounded-error configuration (e.g., `eps`, block, event budget) the engine cannot guarantee `max_err` under the model. This is **not** data corruption.

11.2 Memory rules (ABI)

- The output buffer from `compress` is allocated in the DLL and must be freed via `szy_free_buffer()`.
- `decode-into` writes directly into the user buffer without extra copies (requires a buffer of the correct size).

11.3 Threading policy

- `szy_hw_threads()` returns the number of available hardware threads.
- The `*_ex` API takes `num_workers`:
  - $0 =$ auto,
  - $> 0 =$ forced.
- If the requested number exceeds available: `SZY_E_THREADS`.

11.4 API error semantics

Not all errors returned by the API mean a corrupted file or an implementation bug. In practice:

- `SZY_E_SHA_*`, `SZY_E_INDEX`, `SZY_E_TILE` indicate integrity/stream correctness problems,
- `SZY_E_THREADS` indicates an invalid execution configuration,
- `SZY_E_BOUNDS` indicates that with the current settings the bounded-error cannot be satisfied for the input.

From a systems perspective, `SZY_E_BOUNDS` is desirable behavior: it is better to explicitly refuse compression than to return a stream that violates the declared error bound.

11.5 Selected DLL exports (engine)

Listing 3: Selected DLL functions (include/szy_dll.h)

```
SZY_API int szy_hw_threads(void);
SZY_API void szy_free_buffer(void* ptr);
SZY_API const char* szy_strerror(int code);

SZY_API int szy_peek_shape_v8(const uint8_t* buf, size_t n,
                              int* H, int* W, int* B);

/* decode into */
SZY_API int szy_decompress_into_buffer_ex2(
    const uint8_t* buf, size_t n,
    double* out, size_t out_elems,
    int num_workers, int parallel_min_tiles,
    int* H, int* W);

/* threaded compress (float64/f32/u16) */
SZY_API int szy_compress_buffer_ex(
    double* x, int H, int W, double eps, int B, int num_workers,
    uint8_t** out_buf, size_t* out_n);

SZY_API int szy_compress_f32_ex(...);
SZY_API int szy_compress_u16_ex(...);
```

11.6 DLL function descriptions (what they do and how to use them)

The description below refers to the listing above (selected exports). The goal is to unambiguously explain parameter semantics, memory, and return codes.

`szy_hw_threads()`. Returns the number of available hardware threads (at least 1). Useful for setting `num_workers=0` (auto) or validating that the application does not request too many threads.

`szy_free_buffer(ptr)`. Frees a buffer allocated by the DLL (e.g., `compress` output). **Contract:** this pointer must come from SZY; do not free it by other means (e.g., `free()` in the application) to avoid mixing allocators.

`szy_strerror(code)`. Returns a static string describing the error code. Useful for logs and diagnostics.

`szy_peek_shape_v8(buf,n,H,W,B)`. Reads only the container header and returns dimensions (H,W) and block (B) **without decoding data**. The function verifies container integrity (SHA) just like a full unpack.

`szy_decompress_into_buffer_ex2(...)` **(decode-into).** Decodes a container into a user buffer:

- `out` points to a `double` array of at least `H*W` elements,
- `out_elems` is the number of elements in `out` (not bytes),
- `num_workers`: 0=auto, >0=requested thread count (strict mode),
- `parallel_min_tiles` specifies from how many tiles it is worth switching to parallel mode.

On decoder error, the contents of `out` should be considered invalid.

`szy_compress_buffer_ex(...)`, `szy_compress_f32_ex(...)`, `szy_compress_u16_ex(...)`. Encode a `double`, `float`, or `uint16` matrix into an SZ2D v8 container buffer.

- `eps` is the bounded-error parameter (absolute bound),
- `B` is tile size (e.g., 64 or 128),
- `num_workers`: 0=auto, >0=requested thread count (strict mode),
- `out_buf/out_n` are filled on success; `out_buf` must be freed via `szy_free_buffer()`.

The encoder may return `SZY_E_BOUNDS` if with the given `eps` and internal configuration it cannot maintain the bounded-error guarantee.

12 Build (Windows/MSVC)

Listing 4: Building the DLL (MSVC, current)

```
cmd /c "vcvars64.bat && cd /d D:\projekt\SzKompresor && cl /nologo /O2 /W4 /MD /LD /
    DSZY_BUILD_DLL ^
  src\szy_bytes.c src\szy_varuint.c src\szy_container_v8.c src\szy_tile_v8.c src\szy_buf.
    c src\szy_sha256.c ^
  src\szy_bitplane.c ^
  src\szy_hw.c ^
  src\szy_encode_v8.c src\szy_encode_f32.c src\szy_encode_u16.c ^
  src\szy_decode_v8.c src\szy_parallel.c src\szy_errors.c src\szy_exports.c ^
```

```
/I include /I src ^
/Fe:szytool.dll ^
/link /LIBPATH:static zstd_static.lib zlib_static.lib"
```

13 Engineering tests and benchmarks: methodology

13.1 Artifact identification (DLL SHA256)

Example auto-suite run:

Listing 5: DLL SHA256 (example)

```
dll_sha256 = 02a9664d540776734d91c93b8355723787f140532808f148a685dac46ff12ade
```

13.2 Two test levels: synthetic auto-suite and public benchmark

To get a complete picture, two tools are used:

- **Auto-suite v1 (synthetics + MNIST/Fashion-MNIST)**: older script `compare_all_szy_sz3_zfp_lossl` which generates synthetics, downloads MNIST/Fashion-MNIST, and builds 2D mosaics. Still useful as a stress test and regression.
- **Public benchmark v3 (bench_public.py)**: newer script described below, using open public data: GeoTIFF (satellite image), UCI sensors, synthetic climate data, and medical/astro placeholders.

13.3 Public benchmark (bench_public.py): data and codecs

The `bench_public.py` script:

- uses the `SzyDLL` wrapper (ctypes) to call `szy_compress_*` and `szy_decompress_into_buffer_ex2`,
- uses the same error metrics for all codecs (max_err, RMSE, p99_err),
- tests the following codecs:
  - SZY v8 (DLL),
  - SZ3 (pysz),
  - ZFP (zfpy),
  - Zstd (python zstandard, lossless),
  - Blosc2 (zstd, lossless).
- produces:
  - `compare_all.json` — full results,
  - `compare_all.csv` — all rows table,
  - `report.md` — short textual report,
  - `plots/` directory — PNG plots ratio/max_err per dataset.

Public datasets used in the benchmark.

- **sat_landsat_L8_B4_001002_20160816**: GeoTIFF sample `RGB.byte.tif` from the Rasterio repository (band 1, uint8 cast to uint16), padded size: 768×896.
- **sat_placeholder_2048x2048**: synthetic uint16 gradient 0..65535 (stress-test for lossless).
- **med_placeholder_ct_512x512**: synthetic CT-like uint16 (4 quadrants + noise).
- **astro_placeholder_ccd_2048x2048**: synthetic CCD-like uint16 (noise + "stars").
- **clim_placeholder_temp_720x1440**: synthetic temperature field f32 (lat/lon + sinusoidal modulation).
- **sensor_temp_daily_min**: time series of daily minimum temperatures (jbrownlee), folded into a matrix.
- **sensor_airquality_uci**, **sensor_energy_uci**: UCI repository data (AirQuality, Energy) reduced to 2D NumPy arrays in f64.

Python libraries used (benchmark).

- NumPy — matrix processing, error computations.
- `netCDF4`, `rasterio` — for versions that use real NetCDF/GeoTIFF (currently AVHRR/ocean samples are optional; require files in Unidata repo; if 404 they are skipped cleanly).
- `pandas`, `matplotlib`, `zfpy`, `pysz`, `zstandard`, `blosc2`.

13.4 Auto-suite v1: synthetics + MNIST/Fashion-MNIST (older pipeline)

The MNIST/Fashion-MNIST section (mosaics 2688×2688) comes from the earlier auto-suite and remains conceptually valid, although the newest test set (bench_public) focuses on different data classes (climate/sensor/satellite/CT/astro). MNIST/Fashion-MNIST results are still useful as an image-like scenario and regression (showing that SZY satisfies eps and is competitive with SZ3 on integer-like images 0..255).

14 Results (manual): granule 10880×10880 float64

14.1 Comparison with SZ3/ZFP and lossless baseline

For a 10880×10880 float64 granule and eps=10, B=128 (example values from a manual run):

Table 4: Granule 10880×10880 float64 (eps=10, B=128): codec comparison (example).

Codec	size [B]	ratio	bpp	enc [ms]	dec [ms]	max_err	RMSE	p99_err
SZY v8 (enc_thr=0, dec_thr=0)	56821592	16.666	3.840	2480	1357	9.8	5.658	9.701
SZ3 (pysz), ABS eps=10	53413190	17.730	3.610	5574	3755	10.0	5.769	9.900
ZFP (zfpy), tol=20	114491936	8.271	7.738	5168	3676	7.375	1.370	3.625
Zstd lossless (lvl=3,thr=1)	237364396	3.990	16.042	8030	6696	0.0	0.0	0.0
Blosc2 lossless (zstd,thr=4)	210905017	4.490	14.253	11409	5166	0.0	0.0	0.0

Conclusions (granule).

- SZ3 has slightly better bitrate, but SZY has much faster decode and a simpler API.
- ZFP in this configuration produces a stream about 2× larger (worse bitrate) than SZY/SZ3.
- SZY brings an engine: SHA + index + decode-into, important for transmission and archiving systems.

15 Results (auto-suite v1): synthetics + MNIST/Fashion-MNIST (older experiment)

This section describes an earlier test set and remains useful as additional context. The newest data from the public benchmark is described in Section 16.

15.1 Results on downloaded data (MNIST/Fashion-MNIST): summary table

The table below summarizes selected rows from the earlier `grok_summary.md` for 2688×2688 mosaics (grid=96). The results show that SZY meets `max_err_ok` on these data for `f32/f64` and achieves ratios significantly higher than the lossless baseline.

Table 5: Auto-suite v1 (download): MNIST/Fashion-MNIST 2688×2688 mosaic (grid=96). Ratio comparison and bound satisfaction (selected).

Dataset	dtype	eps	B	SZ3 ratio	SZY ratio	Zstd ratio	SZY OK
Fashion-MNIST	f32	5	128	11.923	11.212	4.909	YES
Fashion-MNIST	f32	10	128	15.658	13.741	4.909	YES
Fashion-MNIST	f32	20	128	21.893	17.338	4.909	YES
Fashion-MNIST	f64	5	128	23.845	22.424	7.753	YES
Fashion-MNIST	f64	10	128	31.316	27.482	7.753	YES
Fashion-MNIST	f64	20	128	43.785	34.675	7.753	YES
MNIST	f32	5	128	17.467	17.891	12.365	YES
MNIST	f64	5	128	34.934	35.783	19.441	YES
MNIST	u16	10	128	–	3.831	6.659	YES

Interpretation (download v1).

- For float (0..255) SZY is very competitive with SZ3 and clearly better than the lossless baseline.
- For `u16` on MNIST, lossless baseline can be better in ratio (high redundancy).

16 Results (public benchmark v3): satellite, sensors, synthetic data

This section summarizes the newest results from `bench_public.py`, based on fully public datasets (GeoTIFF, UCI, jbrownlee, synthetic CT/astro/climate). All results come from a single run:

```
python bench_public.py --dll szytool.dll --outdir out_public \
  --mode all --eps 5,10,20 --block 64,128
```

16.1 Selected datasets and configurations

The summary table below shows the **best** SZY v8 configurations (among eps=5,10,20 and B=64,128) compared to other codecs for several representative datasets. The data comes from `compare_all.csv`.

Table 6: Public benchmark v3: selected datasets, eps, B; ratio comparison (best configurations).

Dataset	dtype	eps	B	Codec	Ratio	max_err	enc [ms]	dec [ms]
sat_landsat_L8_B4_...	u16	20	128	SZY v8	16.02	19.60	9.7	8.8
				Zstd	4.38	0.00	8.8	2.5
				Blosc2	4.50	0.00	10.8	3.1
sat_placeholder_2048x2048	u16	10	128	SZY v8	86.91	9.80	37.0	15.6
				Zstd	725.09	0.00	2.6	2.1
				Blosc2	15.68	0.00	5.2	8.6
med_placeholder_ct_512x512	u16	20	128	SZY v8	6.03	19.57	6.3	4.5
				Zstd	1.84	0.00	8.1	1.6
				Blosc2	1.75	0.00	6.8	1.9
astro_placeholder_ccd_2048x2048	u16	20	128	SZY v8	5.03	19.60	85.2	42.2
				Zstd	1.80	0.00	126.0	29.1
				Blosc2	1.73	0.00	72.7	14.6
clim_placeholder_temp	f32	10	128	SZY v8	1041.17	9.80	10.0	3.7
720x1440 (padded 768x1536)				SZ3	4131.87	9.92	41.1	18.5
				ZFP	23.96	0.49	20.3	12.1
				Zstd	1.68	0.00	17.3	6.1
sensor_temp_daily_min	f64	20	128	SZY v8	618.26	19.57	1.0	0.5
(384x128 pad)				SZ3	1217.39	19.96	3.9	2.3
				ZFP	138.85	2.68	0.2	0.2
				Zstd	44.02	0.00	0.6	0.2
sensor_airquality_uci	f64	20	128	SZY v8	195.99	19.60	5.1	6.7
(4096x128 pad)				SZ3	205.68	20.00	28.1	10.3
				ZFP	91.05	3.06	2.9	2.1
				Zstd	59.90	0.00	3.3	1.7
sensor_energy_uci	f64	20	128	SZY v8	532.27	19.60	4.3	4.8
(4096x128 pad)				SZ3	469.69	20.00	23.4	10.1
				ZFP	125.79	2.74	3.0	2.2
				Zstd	37.80	0.00	4.5	1.8

Key observations.

- **SZY vs ZFP:** in all cases above, SZY achieves clearly *higher* ratio than ZFP at the same eps, while satisfying max_err≤eps.
- **SZY vs SZ3:**
  - On a super-smooth climate field (f32), SZ3 has ∼4× better ratio, but SZY still achieves an impressive $\sim 10^3\times$ and is 3–4× faster.
  - On UCI sensors, SZY reaches comparable or **higher** ratio than SZ3 at the same eps (e.g., sensor_energy_uci: 532x vs 470x) with much shorter encode time.
- **SZY vs Zstd/Blosc2:**
  - On CT/astro/satellite, SZY provides 3–5× better ratio than the lossless baseline at eps 10–20.
  - On sensor data, SZY improves ratio over Zstd by 5–15x.

16.2 Scientific/technical interpretation of results

Based on the table above, several strong and scientifically grounded statements can be made:

- **Bounded-error is actually met.** In all SZY rows, `max_err` is very close to ε (typically 0.98×eps), confirming that the implementation meets the quality contract.
- **SZY beats ZFP under "scientific lossy" conditions.** At the same eps, SZY usually delivers 2–5× better ratio than ZFP, maintaining error control and offering easy integration (container, SHA, decode-into).
- **SZY is competitive with SZ3 on real data.**
  - On sensor data (AirQuality, Energy, daily temperatures), SZY often matches or beats SZ3 in ratio, with noticeably shorter encoder time.
  - On super-smooth fields, SZ3 has a bitrate advantage, suggesting potential benefit from adding a trend module in future SZY versions.
- **Runtime performance makes SZY suitable for online pipelines.** Times on the order of single to tens of milliseconds for matrices of 10^5–10^6 elements make SZY practical both on servers and on laptops/edge devices.

17 Main conclusions (v8 state after tests)

17.1 What works and what is the engine value

- **Integrity**: SHA-256 of container and data works and is reported unambiguously.
- **Threads**: the engine exposes external thread control and validation (`SZY_E_THREADS`).
- **Performance**: fast decode; tile index enables efficient MT decode and decode-into.
- **Position vs ZFP**: in many bounded-error cases SZY provides better ratio than ZFP while maintaining max_err.
- **Real image-like and sensor data**: on UCI/jbrownlee/SST-synthetic, SZY meets the bound and is competitive with SZ3, often faster.
- **Safe bounded-error semantics**: when the current model cannot meet requested `eps`, the engine can explicitly refuse (`SZY_E_BOUNDS`) rather than returning a stream that violates the guarantee.

17.2 What comparisons to SZ3/ZFP teach (without changing the niche)

- On super-smooth fields, SZ3 can achieve much better bitrate — motivating evolution of SZY toward optional trends (the decoder already anticipates `trend_id`; the specification allows it).
- ZFP, despite its mathematically sophisticated construction, often has weaker ratio at the same eps on practical 2D scientific data; SZY fills a gap between "pure ZFP" and heavier SZ3, while also providing a full engine (container, SHA, index).

17.3 Development recommendations (consistent with the tile-based engine philosophy)

- **Optional trend encoding** (plane3/poly) for very smooth data classes: would help where SZ3 currently has an advantage.
- **Expand client-side configuration** (e.g., explicit control over event policy and quantization mode) while keeping simple defaults.
- **Strict vs best-effort mode**:

  - strict — always bounded-error or `SZY_E_BOUNDS`,
  - best-effort — experimental mode where the encoder may slightly exceed eps if the client accepts it.

- **Add `eps_over_range`** as a default metric in reports to better show eps aggressiveness across datasets.

- **CI + CMake + regression tests (golden files) + fuzz decode** as a permanent test suite (decoder robustness verification).

17.4 Scientific potential and practical advantages

Based on the entire documentation and results, the following scientific/technical theses can be stated:

- SZY v8 is a **full-fledged scientific lossy codec** for 2D data with:
  - a hard $L\infty$ bounded-error model,
  - a portable specification (container, tile, SHA),
  - an open C implementation (no exotic dependencies).

- In benchmarks on real data (UCI sensors, time series, GeoTIFF imagery) SZY achieves:
  - ratios in the hundreds (sometimes thousands) at eps around 5–20,
  - encoder/decoder times of single to tens of milliseconds,
  - full error control.

- SZY fills the gap between:
  - "classic" lossless (Zstd, Blosc2),
  - ZFP (excellent mathematically, but no engine),
  - and heavier SZ3 (in terms of configuration/overhead).

  In many scenarios (telemetry, scientific archiving, HPC pipelines with on-the-fly decoding) SZY offers the most balanced compromise.

- Therefore SZY v8 has **real potential as a component**:
  - in climate/ocean codes (NetCDF/HDF5 → SZY),
  - in imaging pipelines (satellite, medical, astronomy),
  - in edge/IoT applications where speed and simplicity matter and a small controlled error is acceptable.

A Appendix A: Artifacts and reproducibility

The auto-suite and public benchmark save:

- `compare_all.json`: metadata + full results,
- `compare_all.csv`: full table,
- `report.md`: textual report,
- `plots/`: PNG (if matplotlib) or alternatively a PGFPlots snippet.

B Appendix B: Minimal compliance specification (must-pass)

A v8-compliant implementation should:

- recognize `magic="SZ2D"` and `version=8`,
- use little-endian for fixed-width types,
- implement varuint as specified,
- verify SHA-256 (`data_sha`, `container_sha`),
- correctly parse the index blob (if `flags&1`),
- decode tile records and validate payload length after decompression.

References

References

[1] National Institute of Standards and Technology (NIST), *FIPS PUB 180-4: Secure Hash Standard (SHS)*, August 2015. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf

[2] P. Lindstrom, *Fixed-Rate Compressed Floating-Point Arrays*, IEEE Transactions on Visualization and Computer Graphics, 2014. https://computing.llnl.gov/projects/zfp

[3] D. Tao, S. Di, F. Cappello et al., *SZ3: A modular framework for scientific data compression.* https://github.com/szcompressor/SZ3

[4] Y. Collet, *Zstandard (zstd).* https://github.com/facebook/zstd

[5] J.-L. Gailly, M. Adler, *zlib Compression Library.* https://zlib.net/

[6] Blosc Development Team, *Blosc2.* https://github.com/Blosc/c-blosc2

[7] Y. LeCun, C. Cortes, C. J. C. Burges, *The MNIST Database of Handwritten Digits.* http://yann.lecun.com/exdb/mnist/

[8] H. Xiao, K. Rasul, R. Vollgraf, *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms.* https://github.com/zalandoresearch/fashion-mnist